

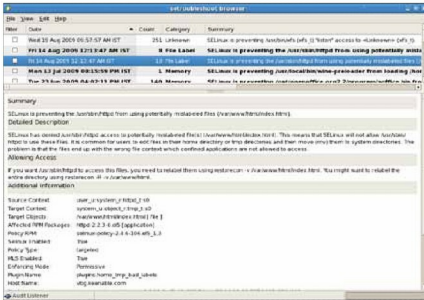


Figure 2: `sealert` can also show errors in a graphical window

Raw Audit Messages

```

avc: denied { getattr } for comm="httpd" dev=hda3
egid=48 euid=48
exe="/usr/sbin/httpd" exit=0 fsuid=48 fuid=48 gid=48
items=0 name="index.html"
path="/var/www/html/index.html" pid=3579 scontext=user_r_u:system_r:httpd_t:s0
sgid=48 subj=user_u:system_r:httpd_t:s0 suid=48
tclass=file
tcontext=system_u:object_r:httpd_sys_content_t:s0 tty=(none) uid=48

```

The above output from `sealert` helps a systems administrator to:

1. Look at the raw log message
 2. Understand why the AVC denial occurred (giving detailed description and summary)
 3. View possible solutions to avoid the AVC denial (by allowing access)
 4. View additional information
- Executing the 'Possible Solution' would generally solve the AVC denial error.

If you execute the solution mentioned below:

```

[root@vbg selinux]# restorecon -v /var/www/html/
index.html

```

...the security context of this file would change to `system_u:object_r:httpd_sys_content_t:s0`. This would remove the

AVC denial and you would be able to browse your HTML page.

For those of us inclined to use the GUI, `sealert` can be initiated in the GUI mode, by giving the following command:

```

[root@vbg selinux]# sealert -b &

```

Please refer to Figure 2, which shows the output for the same error graphically in the browser.

The `setroubleshoot` RPM also includes an init script called `setroubleshoot`. By enabling the `setroubleshoot` script on startup using the `chkconfig` utility, `setroubleshoot` runs as a daemon in the background. Every time an AVC denial occurs, a pop-up appears in the system tray (assuming you are working in the GUI mode). This helps systems administrators to troubleshoot SELinux errors on-the-fly.

At times, you might not be willing to choose the solution

suggested by the `sealert` tool and would rather create your own allow rules in the SELinux policy. For example, I would want my Web server `httpd` to be able to read files of type `tmp_t` rather than restrict it to read only files of type `httpd_sys_content_t`.

In such situations, a new allow rule will have to be added to the Security Policy. This allow rule will permit a subject of type `httpd_t` to read object files of type `tmp_t`.

The above would give you great flexibility in creating your own security environment and not just follow the standard policies that came with the system. At other times, you would require to create security policies for other applications that you may have installed on your system, for example, an Oracle database.

Do you need to build the entire SELinux policy for this? That would appear to be an uphill task for most systems administrators. Therefore, SELinux allows admins to create their own modules.

Instead of modifying the core policy, you can build modules of your own that can be loaded on top of the core policy. In these modules, you can declare your own types and rules.

In the next article in the series, we will look at creating SELinux modules, compiling them and loading them.

I would look forward to your feedback on the articles. Please feel free to drop me a mail for suggestions on improving the content and making it more useful. 

By: Varad Gupta

Varad is an open source enthusiast who strongly believes in the open source collaborative model not only for technology but also for business. India's first RHCSA (Red Hat Certified Security Specialist), he has been involved in spreading open source through Keen & Able Computers Pvt Ltd, an open source systems integration company, and FOSTERING Linux, a FOSS training, education and research training centre. The author can be contacted at varad.gupta@fosteringlinux.com